
CAOS : A Cellular Automaton Sequencer

Professor :

Marteen Kuijk

Authors :

Laurent Béguelin

Teaching assistant :

Ayman Morsy

Tuur Bruneel

1 Introduction

This report is the outcome of a first-year Master's project in Electrical Engineering at VUB, carried out as part of the Sensors and Microelectronics course. The idea grew out of an interest in Conway's Game of Life and, more broadly, in chaotic dynamical systems. The project chosen is a *Generative Cellular Automaton Melody Sequencer*.

Cellular automata (CA) are simple, rule-based dynamical systems capable of producing remarkably rich and unpredictable behaviour from a small set of local update rules. For a one-dimensional elementary CA, each new 32-bit row is generated by evaluating, for every cell, a fixed rule depending only on that cell and its two immediate neighbours in the previous row. Stacking successive generations produces a deterministic yet visually stunning “waterfall”. The generated notes are mapped to an ascending scale, so even chaotic patterns remain pleasant to the ear.

The remainder of this report is organised as follows. Section 2 presents the user manual. Section 3 gives a top-level view of the project, describing the three hardware timers, the physical inputs, and the main data structures held in SRAM. Section 4 documents the register allocation used throughout the assembly source. Finally, Section 5 concludes the report.

2 User Manual

2.1 Overview of the Interface

The instrument exposes four physical input devices and two output devices to the user :

- an **LED matrix** (40×14 , driven through a shift-register chain) used to display the CA waterfall and the moving playhead,
- a **piezo buzzer**, driven by PWM, sonifying the column under the playhead,
- a **4×4 matrix keypad** (16 keys, 0-9, A-F) used to reseed, clear the grid, select the CA rule, select the musical scale, reset, and mute,
- a **two-axis analog joystick**, used to pause/run in RUN mode and move the edit cursor in EDIT mode; its push-button toggles cells on/off, giving full control over the automaton's initial condition,
- a two-position **RUN/EDIT switch** toggling between autonomous playback and manual grid editing.

2.2 Operating Modes

The instrument operates in one of two mutually exclusive modes, selected by the front-panel switch :

RUN mode (switch up).

The beat clock (Timer1) is active : the playhead automatically sweeps across the columns of the matrix at the current BPM. Every time the playhead completes a full sweep, a new CA generation is computed and the waterfall scrolls down by one row. The bit pattern under the playhead is played back as audio through the buzzer.

EDIT mode (switch down).

The playhead is frozen and the beat clock no longer advances it. The joystick instead moves an on-screen edit cursor (Cursor_pos) horizontally/vertically over the matrix, allowing the user to toggle individual pixels of the current generation by hand.

2.3 Keypad Reference

The 16 keys are organised in a 4×4 grid and scanned row-by-row (Section 3.2). Table 1 lists the action currently bound to each key.

Row	Key	Action
1	7	Select CA Rule 30
1	8	Select CA Rule 90
1	9	Select CA Rule 110
1	F	<i>unassigned</i>
2	4	Select Major scale
2	5	Select Minor scale
2	6	Select Pentatonic scale
2	E	<i>unassigned</i>
3	1	Reseed LFSR / current generation
3	2	Clear grid (frame buffer & history)
3	3	<i>unassigned</i>
3	D	Reset (clear grid, restore default BPM, clear mode flags, cursor, playhead)
4	A	<i>unassigned</i>
4	0	<i>unassigned</i>
4	B	Freeze the run
4	C	Panic stop (silence buzzer)

TABLE 1: Function bound to each key of the 4×4 matrix keypad.

2.4 Joystick and Switch

In EDIT mode, deflecting the joystick along the X axis moves the edit cursor left/right. Pushing the joystick toggles the selected cell on or off, allowing full control of the row. In RUN mode, pushing the joystick freezes the run. Figure 1 shows the front-panel layout of the instrument together with the 16-key legend.

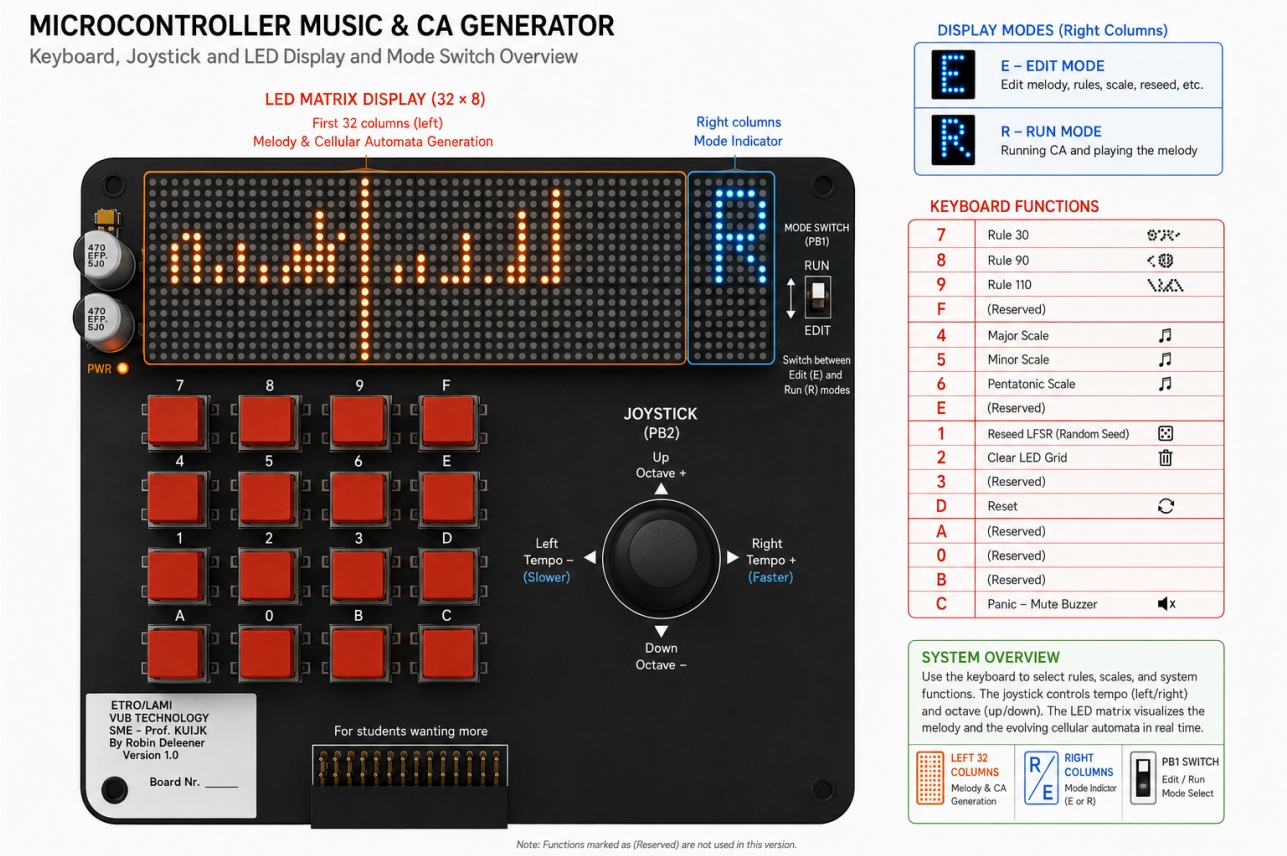


FIGURE 1: Schematic front panel : LED matrix, RUN/EDIT switch, joystick and 4×4 keypad. Image generated by AI [Ope26]

3 Top-Level Architecture

Figure 2 illustrates the hardware connections between the ATmega328P and the external peripherals detailed in Section 2.

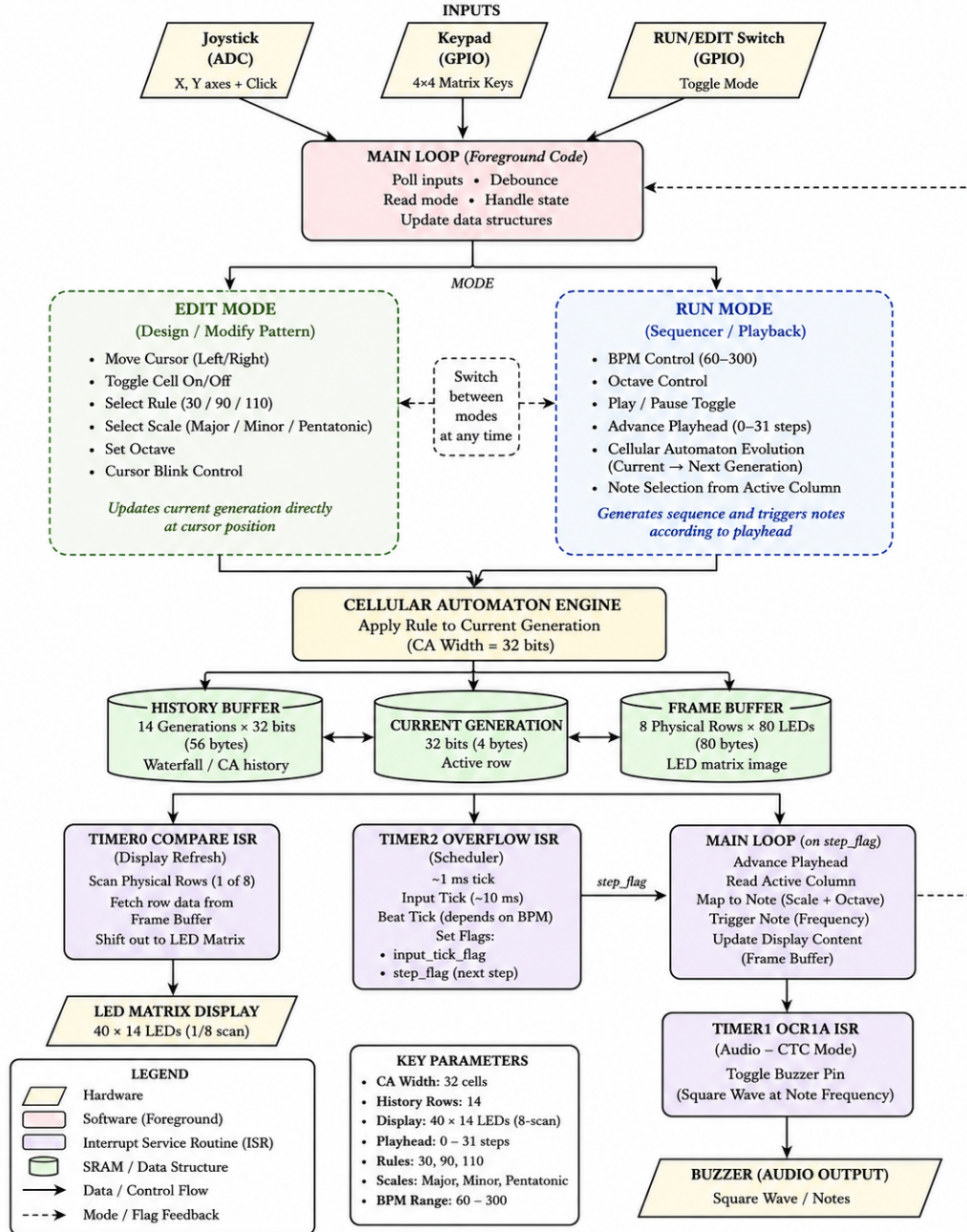


FIGURE 2: Top-level system architecture. The microcontroller processes external inputs and hardware timers to update its SRAM state, dynamically driving the LED matrix and audio output. Image generated by AI [Ope26]

3.1 Timers

Table 2 summarises the hardware timer configurations. They were found reading the data-sheets [15].

Timer	Mode	Registers	Role
Timer0	CTC (WGM01=1) Compare Match A	OCR0A=99 Prescaler : /256	Display Refresh : Timer0_ISR runs at ~625 Hz. It fetches 10 bytes from <code>frame_buffer</code> and shifts them to the LED matrix, scanning one physical row per interrupt without CPU intervention.
Timer1	CTC (WGM12=1) Compare Match A	OCR1A=Variable Prescaler : /8	Audio Generation : Timer1_ISR toggles the buzzer. OCR1A is updated dynamically from the 16-bit <code>Note_Table</code> to generate specific musical pitches.
Timer2	Normal (OVF) Overflow	TCCR2A=0 Prescaler : /64	System Timing : Timer2_ISR generates base timing ticks. It uses software dividers to set the input flag (~10 ms), the cursor blink (~250 ms), and the sequencer tempo (60–300 BPM).

TABLE 2: Hardware timers configuration and their roles.

3.2 Inputs

The firmware polls three physical input devices, gated by Timer2's 10 ms `input_tick_flag` to ensure stable debouncing :

Matrix Keypad (4×4)

Rows (PD4–PD7) are sequentially driven low. Columns (PD0–PD3, internal pull-ups enabled) are read to detect presses. `Read_Keyboard` scans the matrix, and `Check_Buttons` dispatches valid new presses to their respective action routines.

Analog Joystick

Sampled via the ADC on PC0 (X-axis) and PC1 (Y-axis). The ADC is configured for V_{cc} reference and left-adjusted (ADLAR=1) to yield a fast 8-bit result in ADCH. Readings are compared against `ADC_LOW_TH` (64) and `ADC_HIGH_TH` (192) for edge-detected directional inputs. The integrated push-button is read on PB2, programmed according to the AVR Instruction Set [20].

RUN/EDIT Switch

A hardware switch on PB0 (internal pull-up enabled). Read by `Poll_Switch_Mode` to toggle the system between sequence playback (RUN) and single-cell matrix editing (EDIT).

3.3 Main Data

Persistent state is stored in SRAM (Table 3). Computation is decoupled from the display using the `frame_buffer`, which is read asynchronously by the Timer0 ISR. The cellular automaton state is managed via the 32-bit `current_gen` and `next_gen` variables.

Symbol	Size [Bytes]	Contents
<code>history_buffer</code>	56	14 generations of past CA states for the visual waterfall.
<code>frame_buffer</code>	80	Raw pixel data (8 rows $\times$ 10 bytes) read by the display ISR.
<code>current_gen, next_gen</code>	4 each	Current and computed next 32-bit CA states.
<code>playhead_pos,</code> <code>Cursor_pos</code>	1 each	Sweeping sequence playhead (0–31) and edit cursor positions.
<code>bpm_value</code>	2	Current sequence tempo (60–300 BPM).
<code>lfsr_seed</code>	2	16-bit seed for the random number generator.
<code>rule_number_store</code>	1	Active CA rule (30, 90, or 110).
<code>scale_index_store</code>	1	Active musical scale (Major, Minor, Pentatonic).
<code>octave_shift</code>	1	Octave offset for audio frequency mapping.
<code>mode_flags</code>	1	Bitfield for RUN/EDIT, PLAY/PAUSE, and BLINK states.
<code>input_tick_flag,</code> <code>step_flag</code>	1 each	Timer2 request flags for input scanning and sequencer steps.
<code>timer2_divider,</code> <code>beat_tick_limit</code>	1 each	Software counters/limits to derive input and BPM timings.
<code>key_last, prev_js</code>	1 each	Edge-detection states for keypad, joystick direction, and clicks.

TABLE 3: Main SRAM data structures (`.dseg`) and their purpose.

To introduce pseudo-randomness into the cellular automaton, the system employs a 16-bit Galois Linear Feedback Shift Register (LFSR). When the user triggers a grid reseed, the LFSR generates a new 32-bit starting state (computed over two 16-bit steps) to populate `current_gen`. The implementation uses the characteristic polynomial `0xB400`, which corresponds to a maximum-length feedback configuration for a 16-bit register [WM07]. This guarantees a pseudo-random sequence period of $2^{16} - 1$ states before repeating, ensuring a highly diverse and unpredictable starting seed for the automaton.

4 Register Descriptions

To ensure interrupt service routines (ISRs) execute quickly and safely without corrupting main program data, the register file is strictly partitioned using `.def` directives. ISRs are assigned dedicated registers, isolating their execution environment from the main loop’s persistent state. Tables 4 and 5 detail this allocation.

Symbol	Reg.	Used by	Purpose
Column_index	R2	Timer0 ISR	Active physical LED row currently being scanned.
display_tempo	R3	Timer0 ISR	Temporary holding register for SRAM fetches.
beat_counter	R6	Timer2 ISR	Software beat counter driving the sequencer tempo.
debounce_tick	R7	Timer2 ISR	Millisecond divider for the edit-cursor blink rate.

TABLE 4: ISR-exclusive registers.

Symbol	Reg.	Purpose
playhead	R4	Current sequencer step (0–31) being played.
cursor	R5	Edit cursor position on the active grid.
seed_low, seed_high	R8, R9	Low and high bytes of the working LFSR random seed.
rule_number	R10	Active cellular automaton rule (30, 90, or 110).
scale_index	R11	Active musical scale (Major, Minor, Pentatonic).
Octave	R12	Current octave shift offset for the audio synthesizer.

TABLE 5: Main-program state registers.

Beyond these dedicated state registers, specific registers are designated for general-purpose execution and rendering operations : temp (R16) for generic arithmetic, along with px_x (R20), px_y (R21), and px_state (R22) for LED matrix pixel targeting.

Furthermore, R1 is kept permanently cleared (CLR R1) to serve as a fast zero reference (e.g., when clearing buffers), while the Z pointer (R30 :R31) is heavily utilised for indirect SRAM addressing and LPM Flash lookups.

Hardware constants and system parameters are declared via .equ directives at the head of the source file (Table 6). This eliminates “magic numbers” from the code body, allowing display geometry, CA constraints, and timings to be centrally retuned.

Constant	Value	Meaning
NUM_COLS, NUM_ROWS	40, 14	Logical geometry of the LED matrix.
CA_WIDTH	32	Width (in bits) of a single cellular automaton generation.
HISTORY_ROWS	14	Past generations stored for the visual waterfall.
NUM_RULES, NUM_SCALES	3	Number of CA rules and musical scales available.
MODE_*_BIT	0, 1, 2	Bitfield mappings for RUN, PLAY, and BLINK states.
DEFAULT_BPM	120	Default sequencer tempo loaded on startup/reset.
MIN_BPM, MAX_BPM	60, 300	Clamping limits for the BPM parameter.
ADC_LOW_TH, ADC_HIGH_TH	64, 192	8-bit ADC thresholds for joystick direction detection.

TABLE 6: Primary configuration constants (.equ).

5 Conclusion

This project provided an incredible deep dive into the world of microcontrollers and Assembly language. While it required long hours of challenging yet rewarding debugging, it ultimately demonstrated the impressive capabilities of seemingly simplistic hardware. I successfully achieved all the technical goals I set for myself, including precise register management, the configuration of hardware interrupts and timers, and the integration of multiple inputs (matrix keypad, switch, and analog joystick). I would like to sincerely thank my friend Lucas Placentino for his valuable advice, as well as the teaching team of the Sensors and Micro-electronics course for this learning opportunity. Finally, I acknowledge the use of AI tools to assist with code troubleshooting, comprehending complex datasheets, and proofreading this report. The code can be found on my GitHub here : <https://github.com/greenpepperlb/Hardware-Projects>.

Bibliography

- [15] *ATmega328P 8-bit AVR Microcontroller Datasheet* (2015). Datasheet. Microchip Technology Inc. URL : https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf.
- [20] *AVR Instruction Set Manual* (2020). Reference Manual. Microchip Technology Inc. URL : <https://ww1.microchip.com/downloads/en/DeviceDoc/AVR-Instruction-Set-Manual-DS40002198A.pdf>.
- [Ope26] OPENAI (2026). *Image generated with AI*. DALL-E [Generative AI image]. URL : <https://chat.openai.com/>.
- [WM07] WARD, Roy et Tim MOLTENO (2007). « Table of Linear Feedback Shift Registers ». In : *Department of Physics, University of Otago*. URL : <https://www.physics.otago.ac.nz/reports/electronics/ETR2006-1.pdf>.